Semana 02 — Cifrar, firmar y romper la integridad

Grupo 1 · rol Rojo · ejemplo de trabajo

Prof. César Rodríguez | ELECTRONICA | 1 de noviembre de 2026

Curso	CYBERSEGURIDAD — Ciberseguridad Aplicada
Semana	02 de 10
Fase	1 — Fundamentos, criptografía y marco legal
Grupo	1
Rol	Rojo
Entrega	GitHub, carpeta semana_02/
Corte	Domingo 23:59 de la semana correspondiente; lo posterior no se corrige.

1 Tu rol en la práctica

Ejecuta la técnica ofensiva sobre el objetivo autorizado: prepara el entorno, lanza el ataque, registra comandos y payloads, y guarda cada salida con su hash.

- Confirmar el objetivo autorizado y preparar el entorno antes de la primera captura.
- Ejecutar la técnica de la semana en el orden de los pasos y guardar cada salida cruda con su hash.
- Redactar la reproducción: comando, salida esperada y salida obtenida, sin retocar la evidencia.

2 Qué aporta este rol al entregable

Evidencia cruda en semana_NN/evidencia y el apartado de reproducción del informe.

3 Ejemplo paso a paso

El rol Rojo no confia en una primitiva hasta verla fallar: cifra un archivo, altera un byte y comprueba que la integridad se detecta. La evidencia es la salida real de cada paso, no la explicacion.

3.1 Cifrar con AES-256-GCM y guardar clave, nonce y hashes

GCM es cifrado autenticado: produce confidencialidad e integridad en una sola operacion. La clave y el nonce se guardan aparte porque sin ellos no hay reproduccion posible.

```
mkdir -p semana_02/evidencia
head -c 4096 /dev/urandom > semana_02/evidencia/mensaje.bin
python3 - <<'PY'
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
import os
clave = AESGCM.generate_key(bit_length=256)
nonce = os.urandom(12)
datos = open('semana_02/evidencia/mensaje.bin', 'rb').read()
open('semana_02/evidencia/clave.bin', 'wb').write(clave)
open('semana_02/evidencia/nonce.bin', 'wb').write(nonce)
open('semana_02/evidencia/mensaje.gcm', 'wb').write(
    AESGCM(clave).encrypt(nonce, datos, None))
PY
sha256sum semana_02/evidencia/mensaje.bin semana_02/evidencia/mensaje.gcm \
  > semana_02/evidencia/hashes.sha256
```

3.2 Alterar un byte y demostrar que la autenticacion falla

Un solo bit cambiado debe hacer que el descifrado lance excepcion. Si no la lanza, lo que se uso no era cifrado autenticado y el hallazgo se reporta.

```
cp semana_02/evidencia/mensaje.gcm semana_02/evidencia/alterado.gcm
python3 - <<'PY' | tee semana_02/evidencia/integridad.txt
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
p = 'semana_02/evidencia/alterado.gcm'
b = bytearray(open(p, 'rb').read())
b[20] ^= 0x01
open(p, 'wb').write(bytes(b))
try:
    AESGCM(open('semana_02/evidencia/clave.bin','rb').read()).decrypt(
        open('semana_02/evidencia/nonce.bin','rb').read(), bytes(b),
            None)
    print('ERROR: el texto alterado se descifro')
except Exception as exc:
    print('integridad rota:', type(exc).__name__)
PY
```

3.3 Firmar un mensaje, verificar y repetir con el mensaje alterado

La firma cubre el hash del mensaje y la identidad de la clave. Se verifica el original y despues se repite con otro contenido: la segunda verificacion debe fallar y ese fallo es la prueba.

```
openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:2048 \
  -out semana_02/evidencia/firma.pem
openssl pkey -in semana_02/evidencia/firma.pem -pubout \
  -out semana_02/evidencia/firma.pub.pem
openssl dgst -sha256 -sign semana_02/evidencia/firma.pem \
  -out semana_02/evidencia/mensaje.sig semana_02/evidencia/mensaje.bin
openssl dgst -sha256 -verify semana_02/evidencia/firma.pub.pem \
  -signature semana_02/evidencia/mensaje.sig semana_02/evidencia/
     mensaje.bin \
  | tee semana_02/evidencia/firma_ok.txt
printf 'mensaje distinto\n' > semana_02/evidencia/mensaje_alterado.bin
openssl dgst -sha256 -verify semana_02/evidencia/firma.pub.pem \
  -signature semana_02/evidencia/mensaje.sig \
  semana_02/evidencia/mensaje_alterado.bin 2>&1 \
  | tee semana_02/evidencia/firma_alterada.txt
```

3.4 Medir el coste de PBKDF2 y elegir iteraciones con datos

El coste de derivar una clave es una defensa contra la fuerza bruta, pero tambien una condicion de usabilidad. Se mide tres veces cada valor y se reporta la mediana, no la mejor cifra.

```
python3 - <<'PY' | tee semana_02/evidencia/pbkdf2.txt
import hashlib, statistics, time
for iteraciones in (10_000, 100_000, 600_000):
    tiempos = []
    for _ in range(3):
        t = time.perf_counter()
        hashlib.pbkdf2_hmac('sha256', b'clave-de-prueba', b'sal-fija',
                            iteraciones)
        tiempos.append(time.perf_counter() - t)
    print(iteraciones, round(statistics.median(tiempos), 3), 's')
PY
```

3.5 Primitiva, operacion y propiedad garantizada

Primitiva, operacion y propiedad garantizada

Primitiva	Operacion	Propiedad	Si se altera un byte	Resultado observado
AES-256-GCM	AESGCM.encrypt y decrypt	Confidencialidad e integridad	El descifrado lanza excepcion	Falla detectada y registrada
RSA-2048 con SHA-256	openssl dgst -sign	Autenticidad e integridad	La verificacion no valida	Firma rechazada
SHA-256	sha256sum sobre el archivo	Integridad del archivo	El resumen cambia por completo	Cambio evidente
PBKDF2-HMAC-SHA256	hashlib.pbkdf2_hmac	Resistencia a fuerza bruta	No aplica a la medicion	600000 itera unas 60 veces mas

La ultima columna se llena con lo observado en esta practica; si una fila no se ejecuto, se marca como no verificada en vez de suponer el resultado.

4 Qué guardar en el repositorio

El informe enlaza cada archivo de evidencia con el comando que lo produjo y explica, con la tabla, que propiedad cubre cada primitiva y que se rompe al alterar un byte.

```
semana_02/
|-- informe.md
|-- evidencia/
|   |-- mensaje.bin
|   |-- mensaje.gcm
|   |-- clave.bin
|   |-- nonce.bin
|   |-- alterado.gcm
|   |-- integridad.txt
|   |-- firma_ok.txt
|   |-- firma_alterada.txt
|   |-- pbkdf2.txt
|   `-- hashes.sha256
`-- README.md
```

- El fallo de integridad se reproduce con el comando del informe y el archivo alterado.
- La verificacion de la firma alterada aparece en evidencia/ con su salida real.
- La tabla distingue confidencialidad, integridad y autenticidad sin confundirlas.
- Ninguna clave privada ni contrasena real esta en el repositorio.

5 Cómo se corrige

Criterio	Peso
Cifrado y descifrado correctos con verificación de integridad	3,0
Firma y verificación demostradas, incluido el caso alterado	2,5
Lectura correcta del certificado y de la cadena	2,0
Medición de KDF con conclusión sobre coste y seguridad	1,5
Explicación de qué propiedad cubre cada primitiva	1,0
Total	**10,0**

- La clave y el nonce de prueba son desechables; no se reutilizan fuera de la practica.
- {'Un byte alterado tambien puede romper el padding': 'se distingue ese error del fallo de autenticacion.'}

Recuerda

El trabajo es independiente: cada grupo usa su propio repositorio y su propio laboratorio, y la comparación de resultados es en clase. Corte: Domingo 23:59 de la semana correspondiente; lo posterior no se corrige.